AAI-540 · GROUP 4 · DELIVERABLE 2

ML System Operation Validation

Criteo click-through-rate prediction, built and run as a production ML system on Amazon SageMaker

Prashant Khare · Manu Malla · Laxminag Mamillapalli | University of San Diego | October 2026

S1 Ingest & data lake

S2 Features & splits

S3 Train & evaluate

S4 Gate, register & serve

S5 Monitor & retrain

S6 CI/CD & governance

SageMaker Studio Walkthrough

The output is a price, not a label

bid = P(click) × value of a click

A model can rank ads perfectly — high AUC — and still overpay on every bid if its probabilities run high.
Criteo data: 45.8M labelled impressions · 7 days · 13 integer + 26 hashed categorical features.

Where a CTR system fails, and which stage catches it

Probabilities run high Overbid on every impression	S3	**Encoder sees the future** Leak inflates every metric	S2
Malformed upstream feed Model trained on garbage	S1	**Worse model replaces better** Revenue drops silently	S4
Inventory mix shifts Quiet decay before labels arrive	S5	**Unreviewed release** No one owns the decision	S4

Design principles

- **Probability first:** log loss primary, calibration gated.
- **Chronology always:** train on the past, test on the future.
- **Fail fast and cheap:** a data contract runs before any training.
- **Gate twice:** four automated conditions, then a person.
- **Watch data first:** label-free drift signals arrive before clicks do.

MLOps is architecture, not a notebook. Every stage that can fail gets its own control, with an owner and a test.

Four findings that changed the design

DATASET

1 M labelled impressions

7 days of live traffic, in order

39 features: 13 integer, 26 hashed

SOURCE

Criteo Display Advertising Challenge, uploaded as dac.tar.gz to the S3 landing zone

METHOD

Chronological sample in notebook 02, cross-checked on all rows in Athena

Negatives were sub-sampled

Design change

Click rate ≈26% — not the imbalance we assumed, so we neither resample nor reweight.

Missingness carries signal

Feature

Click rate differs when a field is absent, so every numeric keeps an explicit missing flag.

Cardinality spans millions

Feature

One-hot is impossible: hash into 2^20, pool rare levels, add frequency counts.

Ordered in time, no timestamp

Design change

Forces chronological splits; unseen levels later become the earliest drift signal.

Evidence: Athena full-table counts · null rates per column · approx. distinct cardinality · CTR by day · notebook 02 EDA

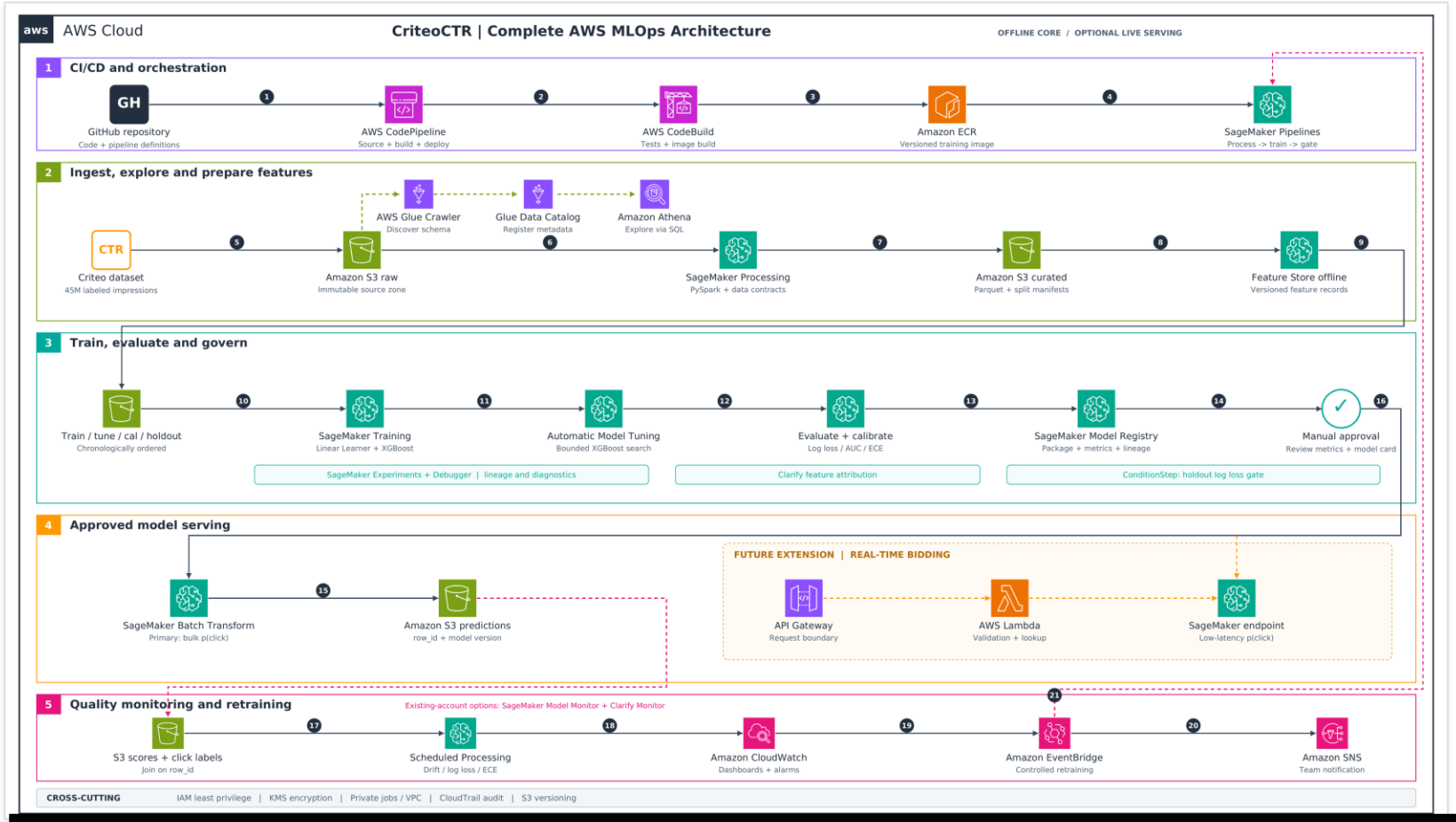
Seven decisions, each with a reason

Framing → Metrics → Model → Split → Features → Release

Decision	Choice	Why it is right here	Status
Framing	Binary classification, probability output	The probability sets the bid	**Built**
Metrics	Log loss primary · NE · AUC · ECE	A constant predictor has AUC 0.5	**Built**
Model	XGBoost vs constant + 2-feature logistic	Trees handle skew and cardinality	**Built**
Split	Chronological 40 / 10 / 10 / 40	Train on the past, predict the future	**Built**
Features	78 inputs; encoder fitted on train only	A leak looks better, not worse	**Built**
Class weight	scale_pos_weight = 1.0	Reweighting inflates every probability	**Measured**
Release	4-condition gate + human approval	No unreviewed model reaches serving	**Built**

Stack: SageMaker built-in XGBoost 1.7-1 · Feature Store · Model Registry · Pipelines · Model Monitor · CloudWatch · SNS

Five stages, 21 numbered hand-offs



S1 CI/CD & orchestration
GitHub → CodePipeline → CodeBuild → ECR → Pipelines

S2 Ingest & features
S3 raw → Athena → Processing → curated → Feature Store

S3 Train, evaluate, govern
Training → tuning → evaluate → registry → approval

S4 Approved serving
Batch Transform primary; real-time endpoint

S5 Monitor & retrain
Scores + labels → checks → CloudWatch → SNS → EventBridge

Cross-cutting: IAM least privilege · KMS encryption · private jobs / VPC · CloudTrail audit · S3 versioning | Step 21 restarts the same pipeline

Running in SageMaker Studio, from one archive

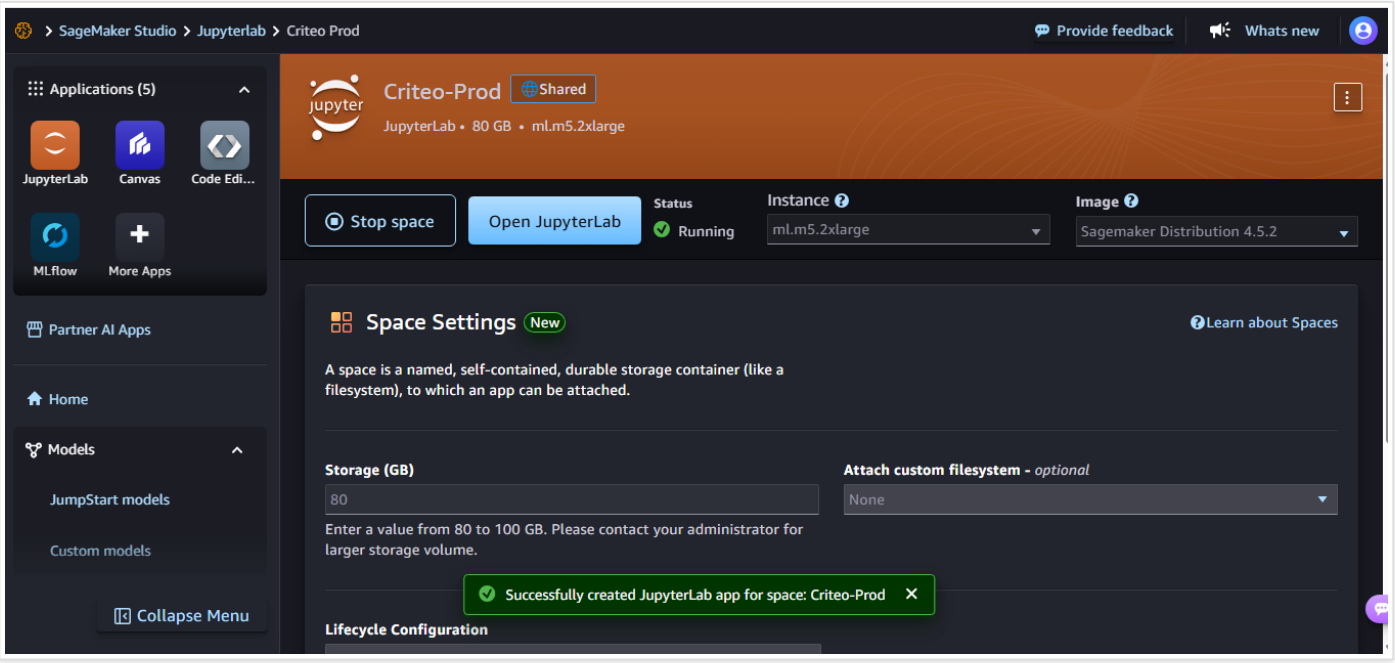
Landing
dac.tar.gz in S3

→ **Stream-extract**
never on disk

→ **Raw zone**
write-once

→ **Curated**
1-in-45, Parquet

→ **Athena**
SQL validation



Criteo-Prod space · ml.m5.2xlarge · 80 GB · SageMaker Distribution 4.5.2 · ap-southeast-2

Byte-checked extraction — Safeguard

Each file must match the size in the archive before it is kept.

Resumable, half the disk — Safeguard

A marker records the source object; only 12.6 GB lands locally.

SDK v2 kernel check — Fix

Studio ships SDK v3; every notebook verifies v2 first.

AWS services: SageMaker Studio JupyterLab · Amazon S3 (landing / raw / curated) · Amazon Athena · AWS Glue Data Catalog

DEMO 1

Three feature groups, label kept apart

FEATURE GROUPS

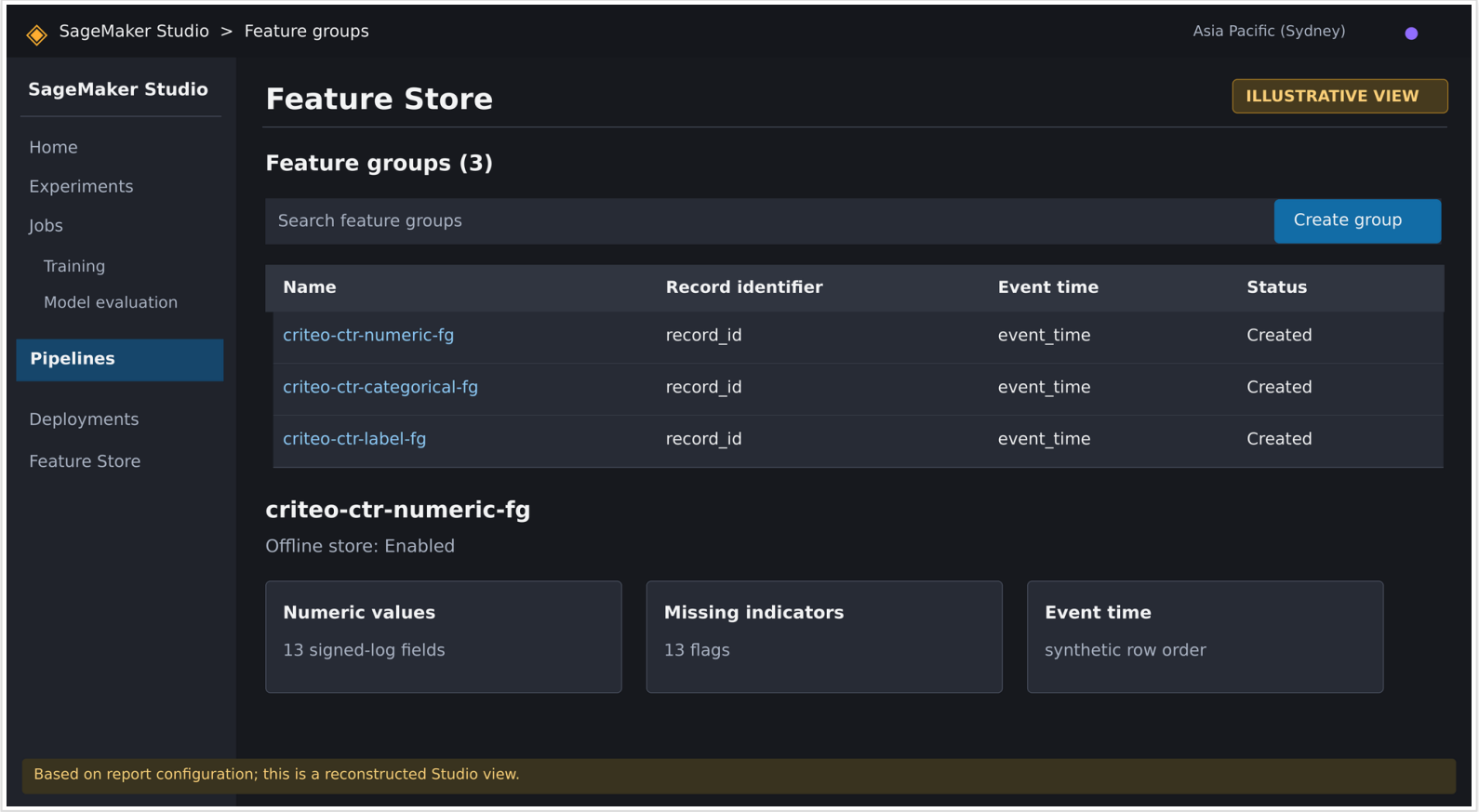
numeric-fg 13 signed-log values + 13 flags
categorical-fg 26 hashed IDs, 2^20 space
label-fg the click, isolated

KEYS

record_id = row position; event_time derived from row order

CHRONOLOGICAL SPLIT

40 / 10 / 10 / 40 train · val · test · production
Encoder fitted on train rows only



Illustrative view from the project configuration — replace with the live Studio capture before recording

Console path: Studio → Data → Feature Store → criteo-ctr-numeric-fg → Details, then Sample query on the offline store

MEASURED

Reweighting the classes overbids every impression

Launcher | Terminal 1 | 11_production_p | 01_data_lake_an | 02_exploratory_c | 04_data_splits.ip | 05_benchmark_r

Code | Notebook | Cluster | Python 3 (ipykernel)

```
assert run_step("evaluate.py", "--model-dir", f"{WORK}/model", "--test-dir", f"{WORK}/out/test",
                "--output-dir", f"{WORK}/evaluation", "--skip-incumbent-lookup",
                "--min-auc", "0.70", "--max-calibration-error", "0.02") == 0
LOCAL_EVALUATION = json.load(open(f"{WORK}/evaluation/evaluation.json"))

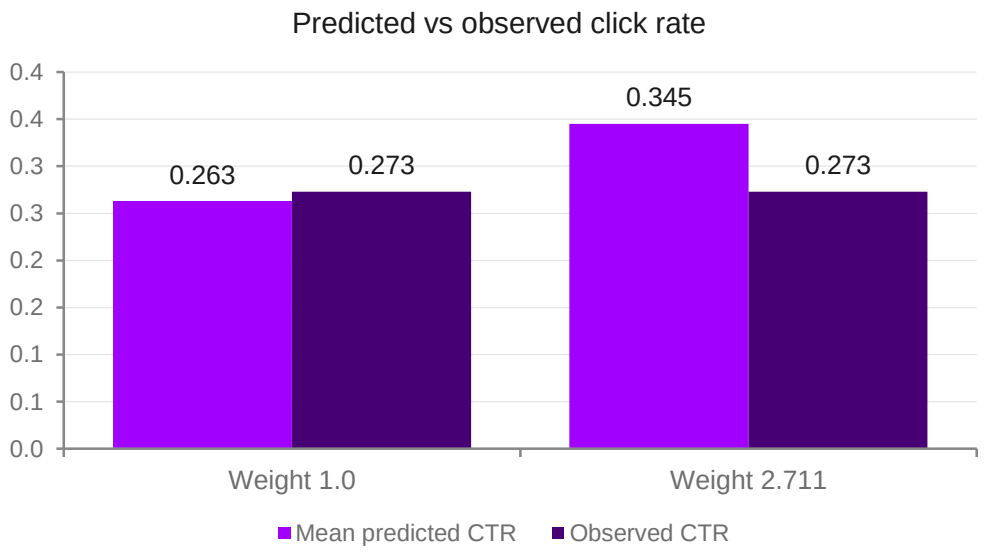
failed = [k for k, ok in LOCAL_EVALUATION["gate_checks"].items() if not ok]
if not failed:
    print("\npre-flight model clears all four production thresholds")
elif failed == ["calibration_within_ceiling"]:
    ece = LOCAL_EVALUATION["binary_classification_metrics"]["expected_calibration_error"]["value"]
    print(f"\nThe gate REJECTS this model on calibration alone (ECE {ece:.4f} > 0.02). That is the")
    print("gate working, not a bug: deep trees fitted to a small, artificially smooth synthetic")
    print("signal produce stepped probabilities. It says nothing about the real model, and the")
    print("threshold is not relaxed to hide it. Section 5 checks both gate branches explicitly.")
else:
    print(f"\npre-flight model fails: {failed}")
```

	scale_pos_weight	rounds	log_loss	mean_predicted	observed_ctr	calibration_error
0	1.000	25	0.5241	0.2630	0.2733	0.0294
1	2.711	77	0.5497	0.3445	0.2733	0.0881

```
artefact: ['model.tar.gz'] <- SageMaker requires exactly this name

$ evaluate.py  (exit 0, 1.3s)
rows 3,000   CTR 0.2733
log loss            0.524133   (base rate 0.586546, incumbent 10.000000)
normalised entropy  0.8936   (<1 beats predicting the average)
```

Notebook 11 pre-flight in SageMaker Studio · synthetic Criteo-shaped data · 3,000 test rows



+26% overbid

Mean prediction 0.345 vs observed 0.273

ECE ×3

Calibration error 0.029 → 0.088

Why: weighting positives by k adds log k to every logit — the same as discarding negatives · the pipeline defaults to scale_pos_weight = 1.0

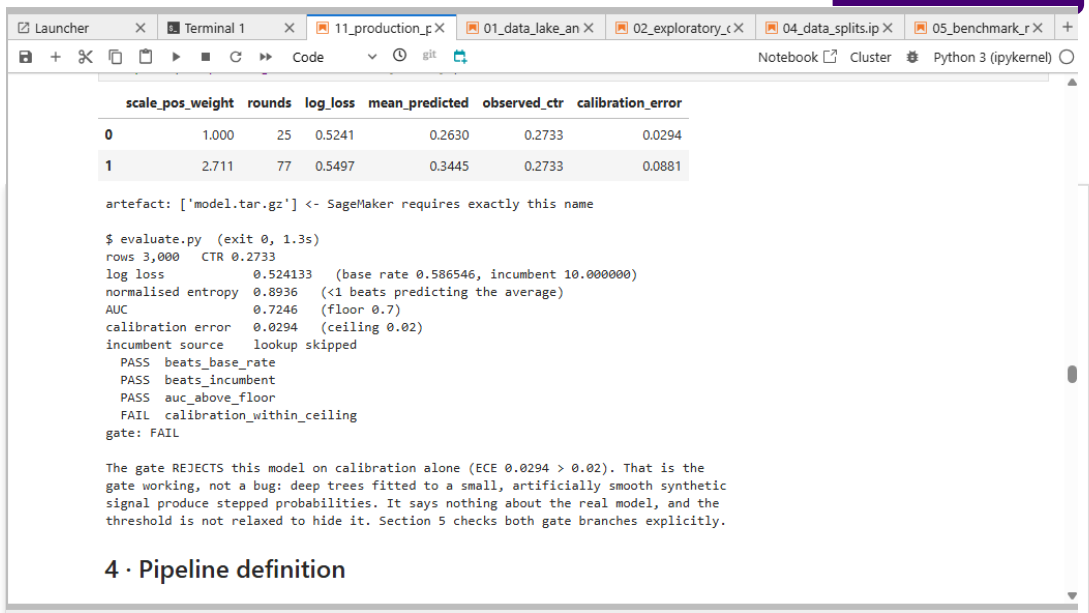
STAGE 4

Four conditions, all must hold

Condition	Threshold	Pre-flight value	Result
Beats base rate	log loss ≤ base rate	0.524 ≤ 0.587	**Pass**
Beats incumbent	≤ approved model	none approved yet	**Pass**
Ranks well	AUC ≥ 0.70	0.725	**Pass**
Calibrated	ECE ≤ 0.02	0.029	**Fail**

Verdict: rejected on calibration.

The incumbent's log loss is looked up in the Model Registry at run time, so a retrain must beat what is serving. The threshold was not relaxed; both branches are proven reachable from the compiled definition.



evaluate.py in SageMaker Studio · synthetic Criteo-shaped data

	Constant click rate	2-feature logistic	XGBoost, weight 1.0	XGBoost, weight 2.94
Log loss	0.5721	0.5451	**0.4756**	0.5435
AUC	0.500	0.653	**0.772**	0.771
Calibration error	0.0051	0.0167	**0.0048**	0.1526
Quality gate	Rejected (AUC)	Rejected (AUC)	**Passes all four**	Rejected (calibration)

Output: evaluation.json — read by the ConditionStep, attached to the registered version, and used as the model-quality monitoring baseline

CI/CD

One pipeline, raw file to registered model

Validate
data contract

Preprocess
splits + encoder

DQ baseline
QualityCheck

Train
XGBoost

Evaluate
evaluation.json

Gate
4 conditions

Pass branch

Pass

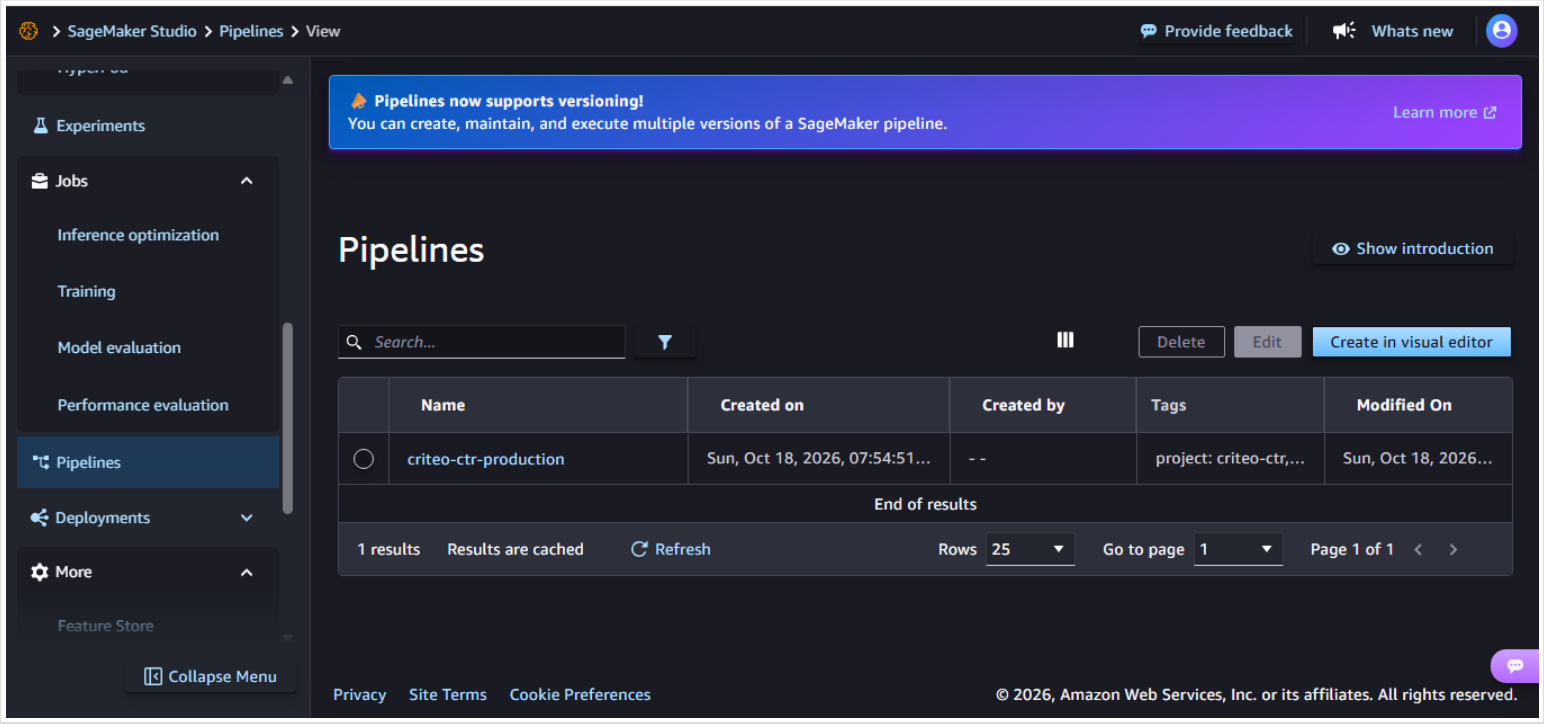
Register as PendingApproval with metrics and drift baselines, then batch-score production.

Fail branch

Fail

A Fail step names the missed threshold. Nothing is registered.

- **18 parameters** — retrain without code changes
- **Retries** on throttling, faults, capacity
- **Caching** 30 days; never on evaluation
- **Lineage** — step code at a content-hashed S3 path

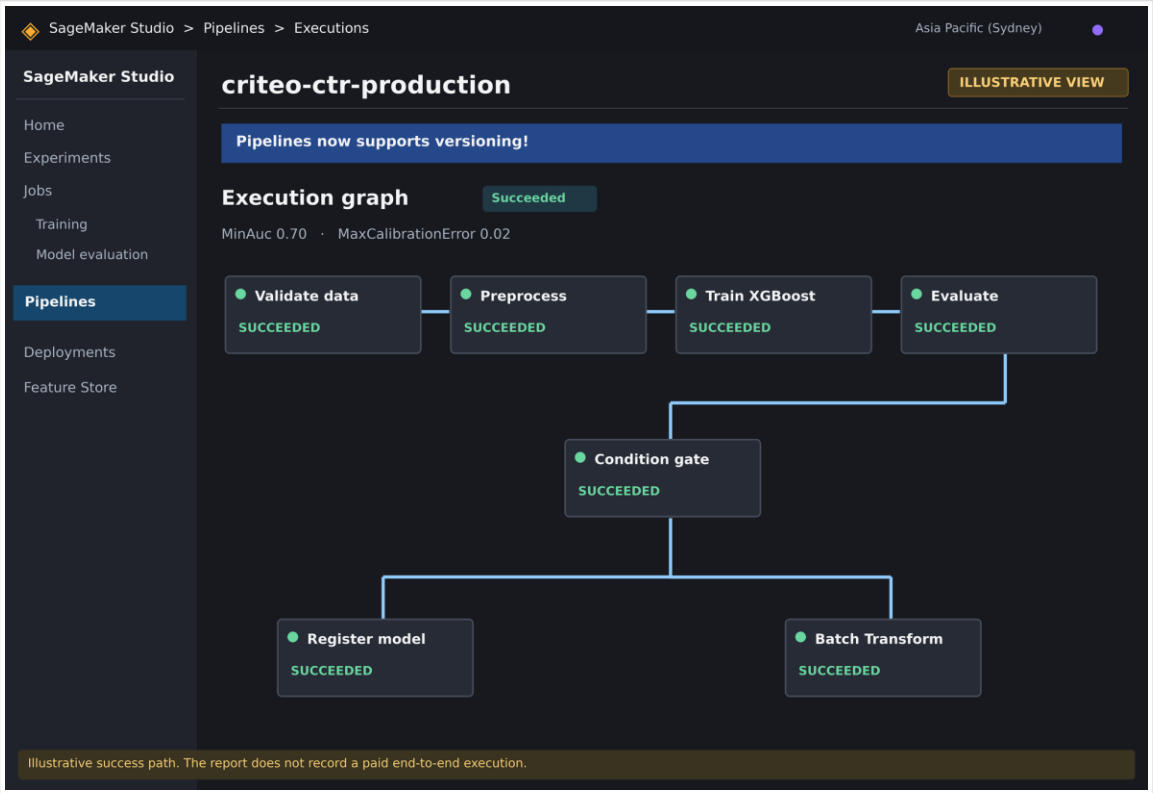


Every push: GitHub Actions — 115 tests · pipeline contract · 3-second end-to-end dry run · notebook parse · lint

DEMO 2

Same code and data, two verdicts

EXECUTION A — SUCCEEDED · production-run-a



EXECUTION B — FAILED AT THE GATE · MinAuc = 0.99



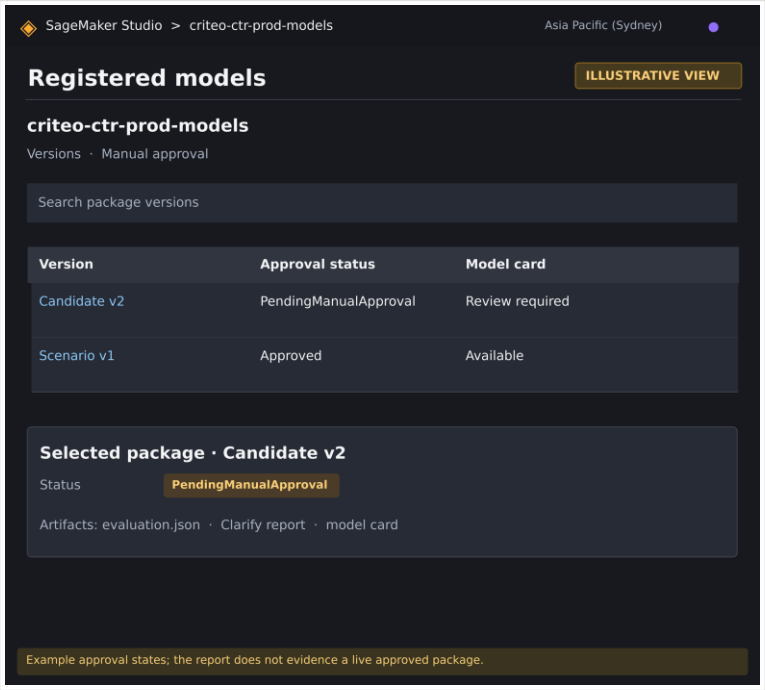
Illustrative view from the project configuration — replace with the live Studio capture before recording

Console path: Studio → Pipelines → criteo-ctr-production → Executions → open each run → Graph, then the Fail step's reason

DEMO 3

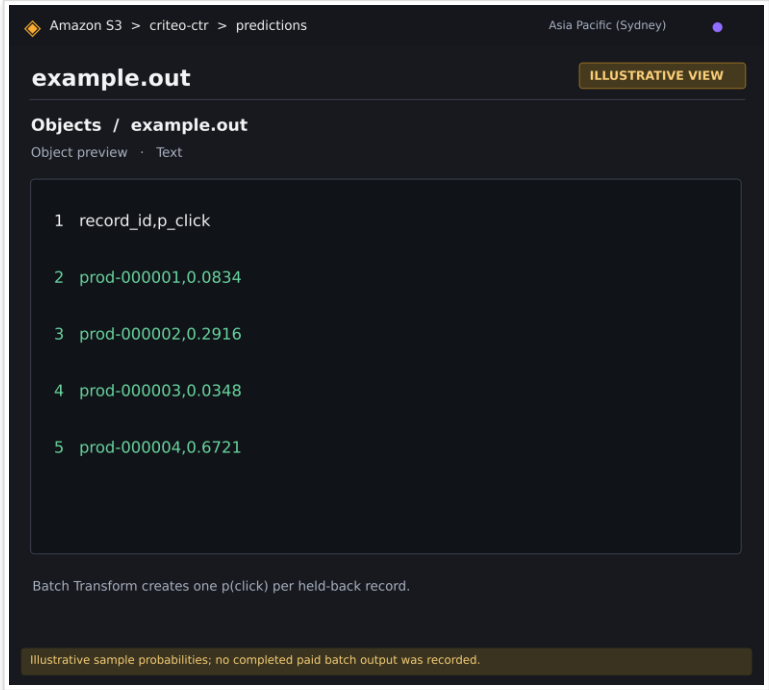
Approved once, served two ways

Model Registry



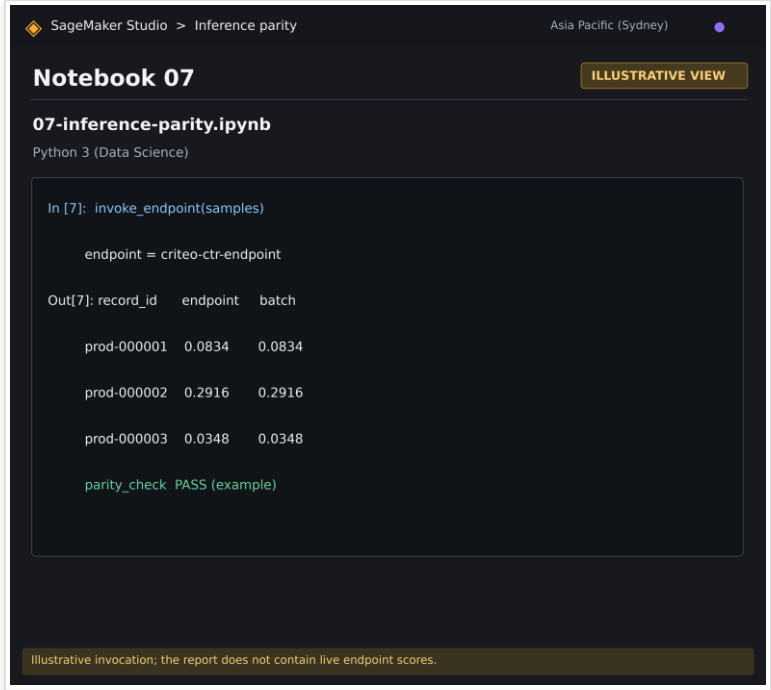
criteo-ctr-prod-models · PendingApproval → Approved

Batch Transform output



One p(click) per production row, by record_id

Endpoint invocation



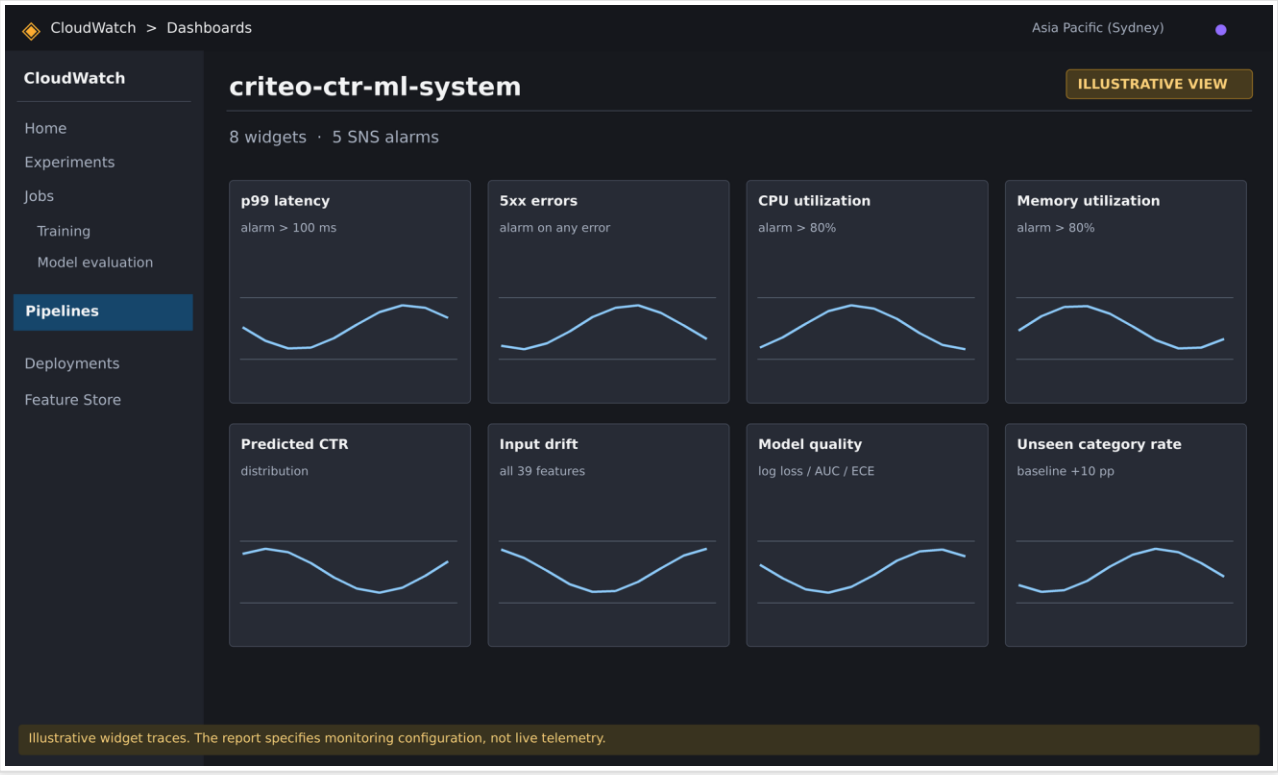
criteo-ctr-endpoint · parity with batch

Illustrative view from the project configuration — replace with the live Studio capture before recording

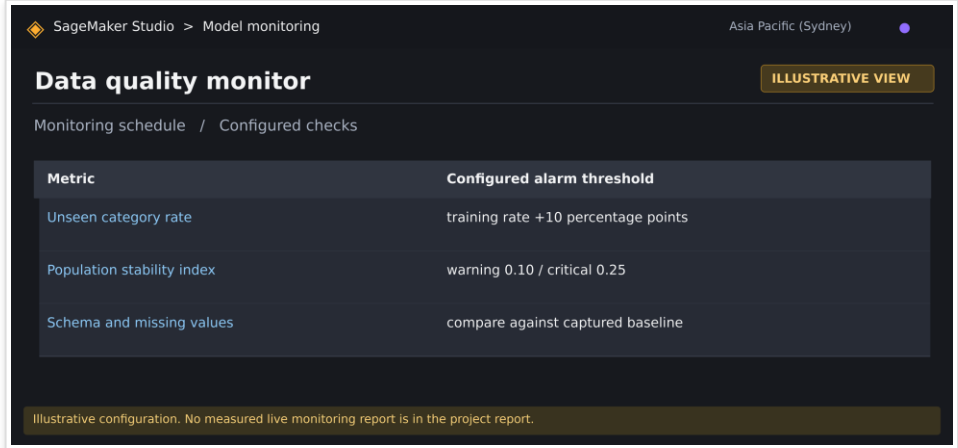
Console path: Models → Registered models → criteo-ctr-prod-models · S3 → criteo-ctr/predictions/ · notebook 07 → invoke criteo-ctr-endpoint

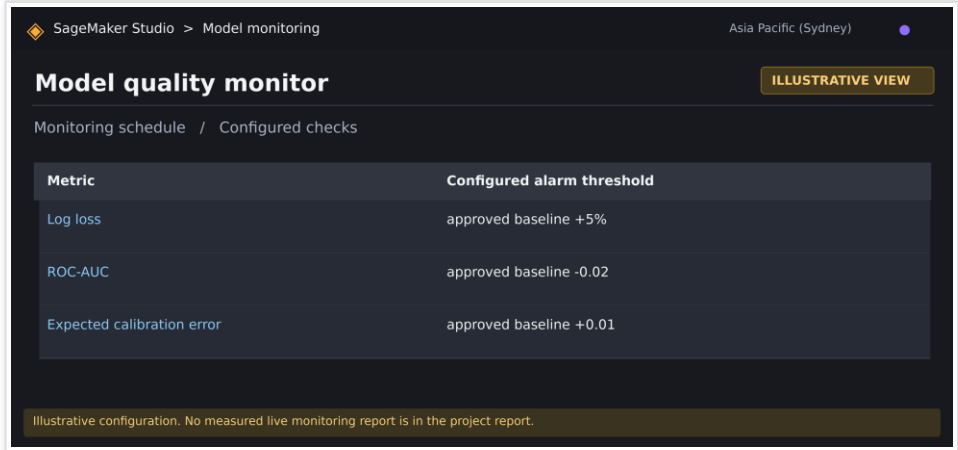
DEMO 4

Watch the data first, the model second



criteo-ctr-ml-system · 8 widgets · 5 alarms · illustrative views — replace with live captures





Unseen-category rate +10 pp vs training

PSI 0.10 warn · 0.25 alarm

Schedules: criteo-ctr-data-quality and criteo-ctr-model-quality (hourly) · alarms: p99 latency > 100,000 µs · any 5xx · CPU / memory > 80%

What broke, and what could break next

SCALABILITY
1.02M-row sample; PySpark at 45.8M rows is designed, not proven

TECHNICAL
Offline gate on one split; gate ECE 0.02 is looser than the monitor's 0.01

ETHICAL
Hashed features can proxy protected traits — no fairness audit is possible; logged data carries selection bias

Difficult issue	What we did	Status
Monitor fired on its own baseline	Absolute rare-level threshold made relative to each column's training rate	**Resolved**
Studio ships SageMaker SDK v3	Own v2 kernel; every notebook checks the version in its first cell	**Resolved**
5 GB default space storage	80 GB space; dac.tar.gz streamed from S3, never stored	**Resolved**
Batch and endpoint "disagreed"	Parity tolerance set below CSV precision	**Resolved**
Retry policy rejected on model steps	API-level policy on model steps; job steps keep both	**Resolved**

Model card states: no protected-class fairness claim on hashed data · calibrated pCTR is not live-traffic CTR · approval is never bypassed on retraining

OUTCOME | CONCLUSION

Reproducible, tested, monitored and gated

115
automated tests, no AWS needed

4
gate conditions + human approval

1.0
class weight, after measuring the overbid

3
monitors: data, model, infrastructure

Future improvements

1. DeepFM / wide-and-deep with learned embeddings
2. Streaming features + Feature Store online store
3. A/B tests with exploration to break selection bias
4. Full-scale PySpark, CodePipeline, weekly retraining on

Thank you

The first verdict in SageMaker was a refusal — evidence that the gate works.

Prashant Khare · Manu Malla · Laxminag Mamillapalli